

Web Application Protection and Automated IP Blocking using ModSecurity, Fail2Ban, and Wazuh SIEM

Project Overview

This project demonstrates an end-to-end **web application security monitoring and response pipeline** by integrating **Nginx + ModSecurity (WAF)** with **Fail2Ban (automated blocking)** and **Wazuh SIEM (detection, correlation, and visualization)**.

The goal of this project is not only to *detect* web-based attacks such as **XSS and malicious HTTP requests**, but also to **automatically block the attacker IP** at the host level and **visualize the full attack-to-response lifecycle** inside Wazuh (and TheHive for SOC visibility).

Compared to a basic WAF-only setup, this project significantly enhances SOC maturity by adding:

- Automated containment (IP blocking)
- Centralized visibility
- Analyst-friendly alerts and correlation

Why This Integration Matters

Before this project:

- Web attacks were only *detected* (403 responses)
- Attackers could repeatedly attempt malicious payloads

After this project:

- Attacks are **detected, correlated, and blocked automatically**
- Attacker IPs are banned in real time
- SOC analysts get **clear visibility** of:
 - The attack
 - The response
 - The blocked IP

This reflects a **real-world SOC workflow** and moves from *monitoring* to *active defense*.

Architecture Overview

Attack Flow:

1. Attacker sends malicious HTTP payloads (XSS / injection attempts)
2. Nginx receives the request
3. ModSecurity (OWASP CRS) detects and blocks the request (HTTP 403)
4. ModSecurity logs the event
5. Fail2Ban parses ModSecurity audit logs

6. Fail2Ban bans the attacker IP using firewall rules
7. Fail2Ban logs the ban action
8. Wazuh Agent collects:
 - Nginx access logs
 - ModSecurity logs
 - Fail2Ban logs
9. Wazuh Manager decodes and correlates events
10. Alerts are visualized in Wazuh Dashboard and SOC platforms (TheHive)

Kali → Nginx → ModSecurity → Fail2Ban → Wazuh → Dashboard / TheHive

Tools & Technologies Used

- **Nginx** – Web server
- **ModSecurity v3** – Web Application Firewall (OWASP CRS)
- **Fail2Ban** – Intrusion prevention & IP blocking
- **Wazuh Agent & Manager** – SIEM, log collection, correlation
- **OpenSearch / Wazuh Dashboard** – Visualization
- **Kali Linux** – Attacker simulation
- **AWS EC2** – Infrastructure

❖ Step-by-Step Implementation

Step 1 – Install Fail2Ban

Purpose: Install the intrusion prevention service used for IP banning.

Fail2Ban is installed and enabled on the web server.

```
root@thehive:/home/ubuntu# sudo apt update
sudo apt install fail2ban -y
sudo systemctl enable fail2ban
sudo systemctl start fail2ban
sudo systemctl status fail2ban
```

Step 2 – Verify Fail2Ban Installation

Purpose: Confirm that Fail2Ban is running correctly.

The Fail2Ban service status is checked to ensure it is active.

```
Synchronizing state of fail2ban.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable fail2ban
● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/usr/lib/systemd/system/fail2ban.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-02-02 14:20:21 IST; 5s ago
     Docs: man:fail2ban(1)
  Main PID: 7600 (fail2ban-server)
    Tasks: 5 (limit: 19161)
  Memory: 27.4M (peak: 27.9M)
    CPU: 224ms
   CGroup: /system.slice/fail2ban.service
           └─7600 /usr/bin/python3 /usr/bin/fail2ban-server -xf start
```

Step 3 – Configure Fail2Ban Jail for ModSecurity

Purpose: Tell Fail2Ban to monitor ModSecurity logs.

A custom jail is created for ModSecurity, pointing to:

- /var/log/modsec_audit.log

Key parameters:

- maxretry
- findtime
- bantime

```
GNU nano 7.2 /etc/fail2ban/jail.d/modsecurity.conf *
[modsecurity]
enabled = true
filter = modsecurity
logpath = /var/log/modsec_audit.log
backend = polling
maxretry = 5
findtime = 300
bantime = 3600
```

Step 4 – Configure Fail2Ban Filter for ModSecurity

Purpose: Define what log pattern represents an attack.

A custom filter is created to match:

- ModSecurity: Access denied with code 403

This allows Fail2Ban to extract the attacker IP from ModSecurity logs.

```
GNU nano 7.2 /etc/fail2ban/filter.d/modsecurity.conf
[Definition]
failregex = ^ModSecurity: Access denied with code .*?\[hostname "<HOST>"\]
maxlines = 10
ignoreregex =
```

Step 5 – Test Fail2Ban Filter with fail2ban-regex

Purpose: Validate that the filter correctly matches attack logs.

The ModSecurity audit log is tested against the filter to confirm matches.

```

root@thehive:/home/ubuntu# sudo nano /etc/fail2ban/filter.d/modsecurity.conf
root@thehive:/home/ubuntu# sudo systemctl restart fail2ban
root@thehive:/home/ubuntu# sudo fail2ban-regex /var/log/modsec_audit.log /etc/fail2ban/filter.d/modsecurity.conf

Running tests
=====

Use   failregex filter file : modsecurity, basedir: /etc/fail2ban
Use   log file   : /var/log/modsec_audit.log
Use   encoding   : UTF-8

Results
=====

Failregex: 7 total
|- #) [# of hits] regular expression
|  1) [7] .*ModSecurity: Access denied with code 403.*\[hostname "<HOST>"\].*
`-

Ignoreregex: 0 total

Date template hits:
|- [# of hits] date format
|  [9] {^LN-BEG}Day(?P<_sep>[-/])MON(?P=_sep)ExYear[ :]?24hour:Minute:Second(?:\.Microseconds)?(?: Zone offset)?
`-

Lines: 190 lines, 0 ignored, 7 matched, 183 missed
[processed in 0.04 sec]

```

Step 6 – Verify Fail2Ban Jail Status

Purpose: Ensure the ModSecurity jail is active and monitoring logs.

Fail2Ban client status confirms:

- Jail is running
- Log file is monitored

```

root@thehive:/home/ubuntu# sudo fail2ban-client status modsecurity
Status for the jail: modsecurity
|- Filter
|  |- Currently failed: 0
|  |- Total failed:    0
|  `-- Journal matches:
`-- Actions
    |- Currently banned: 0
    |- Total banned:    0
    `-- Banned IP list:

```

Step 7 – Configure Wazuh Agent to Monitor Fail2Ban Logs

Purpose: Send Fail2Ban activity to the SIEM.

The following log source is added to the Wazuh agent:

- /var/log/fail2ban.log

This allows Wazuh to ingest ban/unban events.

```
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/fail2ban.log</location>
</localfile>

<localfile>
```

Step 8 – Create Fail2Ban Decoder in Wazuh Manager

Purpose: Teach Wazuh how to understand Fail2Ban logs.

A custom decoder is created matching:

- Program name: fail2ban

```
<decoder name="fail2ban-decoder">
  <program_name>fail2ban</program_name>
</decoder>
```

Step 9 – Create Custom Wazuh Rules for Fail2Ban

Purpose: Generate SOC alerts when IPs are banned/unbanned.

Two custom rules are added:

- **Ban rule** – High severity alert when an IP is blocked
- **Unban rule** – Informational alert when IP is removed

```
<group name="fail2ban,security">
  <rule id="100200" level="10">
    <match>Ban</match>
    <description>Fail2Ban blocked an IP address</description>
  </rule>

  <rule id="100201" level="5">
    <match>Unban</match>
    <description>Fail2Ban unblocked an IP address</description>
  </rule>
</group>
```

Step 10 – Simulate Web Attack from Kali Linux

Purpose: Validate detection and response.

Malicious payloads (XSS) are sent using curl from Kali Linux.

```

(kali@kali)-[~]
$ for i in {1..6}; do
  curl "http://_/_/?test=<script>alert(1)</script>"
done

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.24.0 (Ubuntu)</center>
</body>
</html>
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.24.0 (Ubuntu)</center>
</body>
</html>

```

Step 11 – Verify Fail2Ban Blocking on Victim Machine

Purpose: Confirm automated response.

Fail2Ban status shows:

- Attacker IP banned
- Ban count incremented

```

root@thehive:/home/ubuntu# sudo fail2ban-client status modsecurity
Status for the jail: modsecurity
|- Filter
| |- Currently failed: 1
| |- Total failed:      7
| `-- File list:        /var/log/modsec_audit.log
`- Actions
   |- Currently banned: 1
   |- Total banned:     1 ←
   `-- Banned IP list:  [REDACTED]

```

Step 12 – Visualize Alerts in Wazuh Dashboard

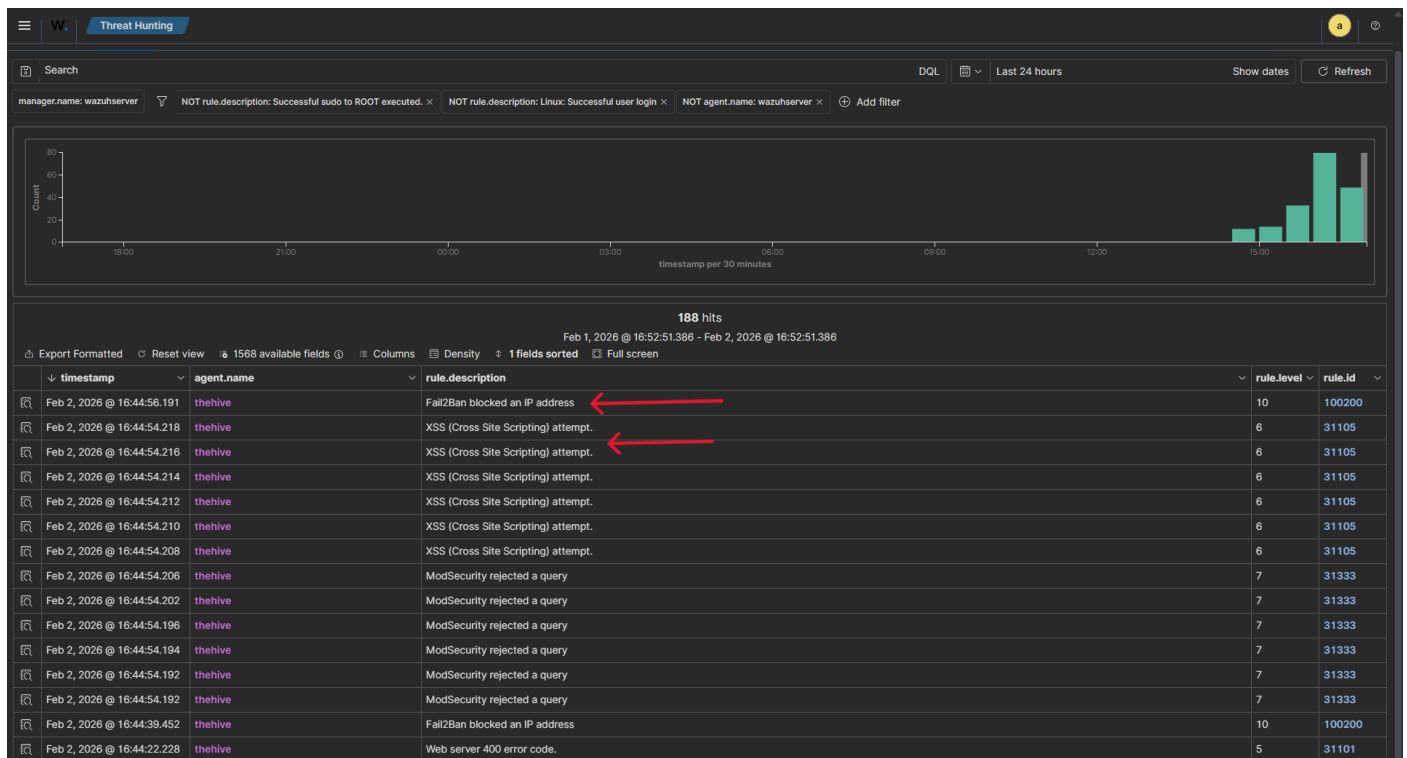
Purpose: SOC visibility and correlation.

In Wazuh Dashboard, analysts can see:

- XSS detection alerts
- Fail2Ban block alerts

- Rule IDs, severity, attacker IP

This clearly shows **attack → response → containment**.



t	agent.name	thehive
t	data.id	403
t	data.protocol	GET
t	data.srcip	
t	data.url	<u>/?test=<script>alert(1)</script></u>
t	decoder.name	web-accesslog
t	full_log	54.226.57.6 - - [02/Feb/2026:16:44:54 +0530] "GET /?test=<script>alert(1)</script> HTTP/1.1" 403 162 "-" "curl/8.17.0"
t	id	1770030894.382572
t	input.type	log
t	location	/var/log/nginx/access.log
t	manager.name	wazuhserver
t	rule.description	<u>XSS (Cross Site Scripting) attempt.</u>
#	rule.firedtimes	6
t	rule.gdpr	IV_35.7.d
t	rule.groups	web, accesslog, attack
t	rule.id	31105
#	rule.level	6
o	rule.mail	false
t	rule.mitre.id	T1059.007
t	rule.mitre.tactic	Execution
t	rule.mitre.technique	JavaScript
t	rule.nist_800_53	SA.11, SI.4

Table	JSON
t _index	wazuh-alerts-4.x-2026.02.02
t agent.id	004
t agent.ip	
t agent.name	thehive
t full_log	2026-02-02 16:44:54,575 fail2ban.actions [150871]: NOTICE [modsecurity] Ban
t id	1770030896.383048
t input.type	log
t location	/var/log/fail2ban.log
t manager.name	wazuhserver
t predecoder.timestamp	2026-02-02 16:44:54,575
t rule.description	Fail2Ban blocked an IP address
# rule.firedtimes	2
t rule.groups	fail2ban, security
t rule.id	100200
# rule.level	10
rule.mail	false
timestamp	Feb 2, 2026 @ 16:44:56.191

Step 13 – SOC Case Visibility (TheHive)

Purpose: Incident response workflow.

Alerts are visible for analysts to:

- Investigate attacker behavior
- Track blocked IPs
- Document incidents

☐	New New 12 minutes ago	M Fail2Ban blocked an IP address	rule=100200 agent_ip= agent_name=thehive agent_id=004 wazuh	wazuh_alert wazuh 000ab6	Observables TTPs	2 0	?	O. 02/02/2026 16:44 C. 02/02/2026 16:44 U. 02/02/2026 16:44
☐	New New 16 minutes ago	M Executable file added - allowed for VirusTotal scan	agent_ip= agent_name=thehive agent_id=004 wazuh rule=100027	wazuh_alert wazuh 3e1e92	Observables TTPs	1 0	?	O. 02/02/2026 16:40 C. 02/02/2026 16:40 U. 02/02/2026 16:40
☐	New New 21 minutes ago	M Multiple XSS (Cross Site Scripting) attempts from same source ip.	agent_id= agent_name=thehive rule=31154 agent_id=004 wazuh	wazuh_alert wazuh d955ce	Observables TTPs	12 0	?	O. 02/02/2026 16:36 C. 02/02/2026 16:36 U. 02/02/2026 16:36

Project Outcome

- ✓ Web attacks detected by ModSecurity
- ✓ Automatic IP blocking by Fail2Ban
- ✓ Centralized visibility in Wazuh
- ✓ Clear SOC alerts with context
- ✓ Real-world defensive security workflow

Skills Gained

- Web Application Firewall tuning
- Fail2Ban custom filters and jails
- SIEM log ingestion and decoding
- Writing custom Wazuh rules
- Attack simulation and validation
- SOC alert correlation and visualization

Conclusion

This project demonstrates a **practical, production-grade SOC use case** where web attacks are not only detected but actively mitigated. By integrating ModSecurity, Fail2Ban, and Wazuh, the system achieves **defense-in-depth**, automated response, and strong analyst visibility.

This setup mirrors real enterprise security architectures and significantly strengthens web application protection.
